Circuit Breaker Pattern

Category: Resilience **One-liner:** Wrap calls to a **remote dependency** (another service, database or API your code calls over the network) in a component that stops calling it once it's clearly failing — failing instantly instead of hanging, and probing periodically to see whether it recovered.

Origin: Popularised by **Michael Nygard** in his book *Release It!*

Part 0 — Concepts you need first

What is a thread, and why do we care?

A **thread** is one line of execution inside a program. A **web server** (the program accepting HTTP requests) keeps a **thread pool** — a fixed set of, say, 200 threads — and assigns one per request, returning it to the pool when done.

The critical property: a thread waiting on a network call does nothing else. It's blocked, occupying a slot. With 200 threads all blocked on a slow service, the server accepts no requests at all — for *anything*. It looks dead though its own code is perfect. That one fact is why the Circuit Breaker exists.

What is a timeout?

The maximum wait before giving up. **Timeouts are necessary but not sufficient**: they stop you waiting *forever*, but every call to a dead service still burns its full timeout. 100 requests/s against a dead service with a 30 s timeout = 3,000 threads tied up. Timeouts bound each wait; the breaker prevents the wait altogether. (Timeouts, retries, backoff and jitter in general: § Timeouts and retries.)

What is a "cascading failure"?

One component's failure causing the failure of those depending on it, like dominoes. **The components that fail second were never broken** — they died waiting on something that was.

What is a fallback?

A substitute response when the real one is unavailable — earlier cached data, a sensible default, a degraded feature. It turns a hard failure into a mild one.

What does "fail fast" mean?

Returning an error *immediately* rather than after a long wait. Often better for users: an instant "this feature is unavailable right now" beats two seconds of waiting followed by an error.

Part 1 — The metaphor

A home **electrical circuit breaker trips** on a dangerous overload, cutting the circuit to prevent fire. You later flip it back; if the fault persists it trips again. Two features carry over:

1. **It cuts the connection to prevent wider damage** — lose one circuit, not the house.
2. **Restoration is a deliberate test** — flip it back and see.

A software breaker trips to stop calls to a failing service, so the system **fails fast and degrades gracefully** instead of grinding to a halt.

Part 2 — The problem

Distributed applications constantly call remote things — **microservices**, databases, external **APIs**. Any call can fail or be slow, and if the application keeps calling an unresponsive service, **requests pile**

up, threads hang, and the failure cascades into a wider outage.

Cascading failures, step by step

Service A depends on B. B goes down or — worse — becomes very slow.

1. Calls from A to B **hang until they time out**
2. During each wait A's threads and memory are **tied up doing nothing**
3. This happens across many threads at once
4. A **runs out of threads or memory**
5. **A now fails**

> **A is broken although nothing is wrong with A's own code** — its logic and database are fine. It died of waiting. Preventing that is the entire purpose of the pattern. And it propagates: whatever calls A now waits on A, so one slow service at the bottom can take down everything above it.

> **Slow is worse than down.** If B refuses connections instantly, A gets an immediate error and moves on. If B accepts and takes 30 s, A's threads are held hostage.

Unresponsive services and wasted work

Long timeouts on slow services delay users and block threads. **Blindly retrying a known-failing operation** wastes CPU and **overloads the failing service further** — dozens of components retrying against an already-struggling database only make it worse.

> The vicious circle: slow because overloaded → clients time out and retry → retries add load → slower → more retries. Client traffic alone can stop a service from ever recovering.

When a fault won't resolve quickly (network outage, crashed service), **continuously retrying is pointless** and prevents graceful handling. **What's needed:** detect failure quickly and stop sending traffic *before* it drags down the whole system.

Part 3 — The solution

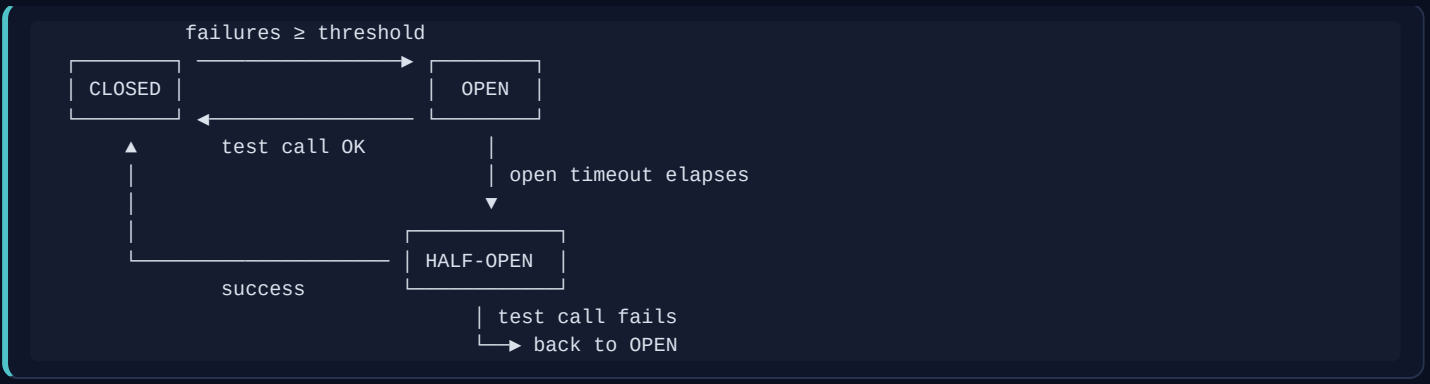
Wrap calls to an external service in a **Circuit Breaker** that monitors their outcomes and stops making them once failures cross a threshold. When it trips, two things happen — the second is easy to overlook:

1. **Your application stops waiting.** Calls are rejected immediately; threads stay free for work that can succeed.
2. **The failing service stops receiving load** — giving it **a chance to actually recover**, breaking the vicious circle.

The pattern prefers **fast failure and graceful degradation** over long waits and resource exhaustion.

Part 4 — The three states

The core of the pattern, and what interviewers ask about.



> *Naming warning:* **CLOSED means traffic flows**, **OPEN means traffic is blocked** — an electrical circuit: a closed loop carries current, an open one is broken.

CLOSED — normal operation

Traffic flows; the breaker **monitors recent failures**. If they exceed a **failure threshold** within a time window, it goes Open. **Successful calls generally reset the failure counter**, so a **transient** error (a glitch that clears itself) doesn't trip it — you want to distinguish "one blip" from "genuinely broken".

OPEN — tripped

Calls are **blocked immediately** — error or fallback **without attempting the real operation** — for a configured **timeout period** (the open interval). Purpose: avoid useless load, and give the service time to recover.

HALF-OPEN — trial mode

After the open timeout, **a limited number of test requests** go through ("Are you OK now?"):

- **Test succeeds** → likely recovered; reset to **Closed** (usually clearing counters)
- **Test fails** → **immediately back to Open**; the timeout restarts

Only a few requests pass in half-open — deliberately, to avoid blasting a recovering service. Often one success closes the circuit; some implementations require several consecutive successes.

> Half-open is the cleverest part: without it you'd need a human to reset the breaker, or a blind reopening of full traffic that instantly re-kills a service just getting back up.

The per-call logic

- **Closed** → attempt the call, record the outcome
- **Open** → skip the call, return failure or fallback immediately
- **Half-Open** → allow the call (perhaps limiting **concurrency** — calls in progress at once), then adjust state by the result

The breaker is a **safety valve**: a failing component can't drag down the rest.

Part 5 — Integration: it's additive, not invasive

A breaker **requires no system overhaul** — it's a protective layer around existing calls, leaving service logic untouched; every request goes *through* it. **Rollout advice:** introduce it gradually, starting with **critical services whose failure would significantly impact the system**, then expand as benefits show.

Part 6 — Configuration

Parameter	What it controls
Failure Threshold	Failed requests before switching Closed → Open
Timeout Period	How long it stays Open before Half-Open — the recovery time given to the service
Number of Requests in Half-Open	How many trial requests test recovery

They must **balance** protecting the service, giving it recovery time, and keeping it available. The right values depend on the nature of the service and system, its expected load, and how critical it is.

These parameters shouldn't be static

They **can and should be adjusted dynamically** with system state. Under high demand you might **increase the failure threshold** (tolerate more failures before tripping) and **reduce the timeout period** (attempt recovery sooner, to serve more requests at peak).

Part 7 — Worked example (Java)

A basic implementation with no libraries, illustrating the core algorithm:

```java
public class CircuitBreaker {
    private enum State { CLOSED, OPEN, HALF_OPEN }

    private State state = State.CLOSED;
    private int failureCount = 0;
    private final int failureThreshold;
    private final long openTimeout;      // how long to stay open before half-open
    private long lastFailureTime = 0;

    public CircuitBreaker(int failureThreshold, long openTimeout) {
        this.failureThreshold = failureThreshold;
        this.openTimeout = openTimeout;
    }

    public synchronized <T> T call(Callable<T> remoteCall) throws Exception {

        // 1. If OPEN, has the timeout passed?
        if (state == State.OPEN) {
            long timeSinceFailure = System.currentTimeMillis() - lastFailureTime;
            if (timeSinceFailure < openTimeout) {
                // Still within the open window — fail fast, don't call the service
                throw new IllegalStateException(
                    "Circuit is OPEN. Failing fast without calling remote service.");
            } else {
                // Timeout has passed — allow one trial call
                state = State.HALF_OPEN;
            }
        }

        // 2. Attempt the call (allowed when CLOSED or HALF_OPEN)
        try {
            T result = remoteCall.call();
            // Success: reset the counter and close the circuit
            failureCount = 0;
            state = State.CLOSED;
            return result;
        } catch (Exception e) {
            // Failure: count it and record when it happened
            failureCount++;
            lastFailureTime = System.currentTimeMillis();
            if (failureCount >= failureThreshold) {
                state = State.OPEN;   // trip
            }
            throw e;   // propagate (or return a fallback here)
        }
    }
}
```

Walking through the mechanics

- **The state check comes first.** Open and within the timeout → throw immediately; **the remote service is never contacted** — no network call, no blocked thread, microseconds instead of seconds.
- `Callable<T>` **makes it generic** — Java's interface for "an operation that returns a value and may throw", so the breaker wraps *any* operation: an HTTP call, a database **query**, anything.
- `synchronized` **handles concurrency** — without **synchronization** (threads taking turns on shared data) two threads could both read `failureCount` as 2 and lose a count. (Also its main performance weakness — below.)
- **Success resets everything** — `state = CLOSED` unconditionally: from Half-Open that's recovery; from Closed it clears accumulated failures.
- **Failure records the time** — `lastFailureTime` is what the Open check measures against.

Using it

```
CircuitBreaker cb = new CircuitBreaker(3, 10000);  // trip after 3 failures, stay open 10s

try {
    String data = cb.call(() -> unreliableService.getData());
    // use data...
} catch (Exception e) {
    useCachedData();   // fallback
}
```

1. `getData()` fails three times → circuit opens
2. Any call in the next 10 s throws **immediately**, without calling `getData()`
3. After 10 s the next call goes through (Half-Open)
4. Success → Closed, normal operation; failure → re-opens for another 10 s

What this example simplifies

- **A naive counter, not a rolling window.** Three failures trip it *whenever* they happen — even spread over an hour of perfect traffic. Real implementations use a **rolling time window** ("more than 50% of calls in the last 10 s failed").
- `synchronized` **serialises every call.** Every request takes the same **lock**, so the breaker becomes a **bottleneck** under load. Production libraries use **atomic operations and non-blocking synchronization** (indivisible steps needing no lock).
- **All exceptions count as failures.** A **validation error** (rejected input) would trip the breaker though the service is healthy. Real breakers distinguish exceptions that mean the *service* is unwell.
- **No fallback built in.** Most frameworks let you register a fallback function directly.

Part 8 — Performance implications

The breaker adds a negligible per-call overhead (check state, update counters). Without it a failing call ties up a thread for **seconds** until timeout; with it, the failure is handled in **milliseconds**. For extremely sensitive paths there are **asynchronous** calls or optimised breaker logic, but the benefit far outweighs the cost.

Tuning the failure threshold

- **Too low** → trips on every minor glitch: needless interruptions and **flapping** (rapid on/off cycling)
- **Too high** → waits too long, failing to protect until many errors have occurred

Tripping after **1** failure is too sensitive; after **100** too late. Pick a threshold that **separates normal transient failures from a real problem**. In high-traffic systems prefer a **percentage-based threshold** over an absolute count, so spikes don't trip it.

Tuning the open timeout

- **Too short** → goes Half-Open **before the service has recovered**, fails again: noisy open → half-open → open **flapping**
- **Too long** → refuses calls after the dependency is healthy again, degrading UX

It should roughly match **how long the service might need to recover**.

Finding the right values

Monitor **how often the breaker opens, how long it stays open, and whether it trips too often**; feed **metrics** (failure counts, open events, half-open trial outcomes) into **dashboards** and adjust.

Part 9 — Best practices and trade-offs

Fallbacks. When open, return cached data, a default, or a simpler alternative rather than just an error — this is what makes the system **degrade gracefully instead of failing completely**. Many frameworks take a fallback function.

Combine with retries and timeouts. The breaker doesn't replace other **resilience** patterns: you still need **timeouts**, and may **retry** a few times on certain failures before counting one failure — retry handles a transient glitch, the breaker stops you after repeated failures that indicate a real outage. How the two interact (and their ordering): Retry Part 4 and pitfall 6. **Bulkheads** and **rate limiting** complement both.

Monitoring and alerts. A tripped breaker usually means a **downstream** service (one you call) is unwell: log/emit an event on every open and close, and alert **on-call** engineers or trigger automated recovery.

> The hidden danger: the system quietly protects itself, users see a degraded feature, and **nobody realises anything is broken** — the breaker suppresses exactly the error flood that would normally raise the alarm.

Thread-safety. Counters and state transitions **must be thread-safe** — two threads mustn't both decide they're "first" to open the circuit. Use atomic counters, synchronized sections, or a library (high-performance ones use non-blocking atomics).

Avoiding misuse. Use breakers on **potentially unreliable or high-latency** calls (network calls); there's **no benefit** on in-memory operations unlikely to fail. Be cautious where failures are very local or self-correcting — **an aggressive breaker constantly tripping on ephemeral (very short-lived) errors does more harm than good.** Tune it to your reliability needs.

Part 10 — Use cases

Microservices architecture

E-commerce with user management, product catalog, cart and order processing; ordering depends on the catalog (availability) and user management (credentials). If the **catalog starts failing**, without a breaker the failures propagate into ordering — degraded UX or total outage. With one, ordering detects it quickly, stops calling, and falls back to a **cached product list** or fails gracefully with a clear message.

External API integration

External APIs are **outside your control**: downtime, latency spikes, rate limits. A weather app pulling from several weather APIs puts a breaker on **each**; a failing or slow one is isolated, and the app **switches to another API** or degrades until it recovers.

Database access

- **Reads:** on rising error rate or latency the breaker trips and **redirects reads to a read replica** (a synced read-only copy) **or a cache**, keeping the service available.
- **Writes:** possible **with more caution** — the breaker could temporarily **buffer writes** (queue them to apply later), but **data consistency is at risk** and the buffer can itself become a bottleneck.

Part 11 — Real-world examples

Netflix and Hystrix. Netflix implemented the pattern in its **open-source** library **Hystrix**: a slow or failing **backend** endpoint is isolated, stopping the cascade and returning fallbacks.

> The canonical illustration: **if the recommendations service is down, Netflix still streams video** — it omits that row or shows a default. A non-essential dependency failing must not take down the essential function.

E-commerce product pages. If the **reviews service** fails, its breaker trips and the page **skips reviews** ("Reviews are unavailable at the moment") while details and pricing load normally — **graceful degradation**. Without it the page might hang or crash fetching reviews. Same for payment gateways and shipping calculators: fall back quickly ("Sorry, we can't process payments right now") rather than leave users waiting.

Cloud and service meshes. Service meshes provide breakers **at the infrastructure level** (Sidecar), and an **API Gateway** can apply breaker rules per backend route — rejecting an erroring backend briefly, protecting it and freeing the gateway for other routes.

Enterprise integration. Any external integration — **SOAP** or **REST** API (SOAP: XML messages with a strict contract; REST: plain HTTP methods on URLs), database, messaging system — benefits. **Middleware** calling a slow **legacy** SOAP service uses a breaker to **avoid tying up all its worker threads**. In **reactive programming** and **stream processing**, breakers support **backpressure** (a slow receiver signalling the sender to slow down): if a **downstream** consumer is slow or failing, the **upstream** producer trips and stops feeding it until it recovers.

Libraries: Hystrix (Java, now in maintenance mode), **Resilience4j** (Java), **Polly** (.NET), plus built-in support in cloud platforms and service meshes.

Part 12 — System design example: distributed social media platform

User Management, **Post Management**, **Feed Generation** and more, each on multiple **nodes** for **high availability** and **load balancing**. Breakers apply:

- **In front of User Management** — Post Management and Feed Generation detect when it struggles and **reduce load** on it by degrading: **cached user information** or **simplified post feeds**.
- **Between Post Management and its database** — on DB problems, serve **cached posts** or **temporarily stop accepting new posts**.
- **In front of the external email service** for notifications — a failing email provider can't affect the rest of the system.

Distributed breaker state

Option: a **shared breaker per service stored in a distributed cache** (e.g. Redis, readable by all nodes), so a trip on one node trips it on all — **balanced against the complexity and latency** of consulting that cache on every call.

> Per-node breakers: each node burns its own failures learning the same lesson. Shared state: one node's discovery protects everyone — but every call now depends on a cache lookup, and the cache is one more thing that can fail.

Monitoring

Monitor all breaker states and alert on trips for quick remediation.

> **The goal is not to eliminate failures** — they're inevitable — but to **manage them**, preventing them from cascading into system-wide outages.

Part 13 — Common beginner misconceptions

- **"Open means working."** The most common mix-up: Open = blocked, Closed = flowing.
- **"The circuit breaker retries for you."** No — it *stops* calls. Retry is a separate pattern; they're used together but do opposite things (retry tries harder, the breaker gives up sooner).
- **"It fixes the broken service."** It protects the *caller* and gives the service breathing room; someone still fixes the real problem — hence alerting.
- **"One breaker per application."** One **per dependency**. Shared, a failure in reviews would block your calls to payments.
- **"If I have timeouts I don't need this."** Timeouts bound each wait; the breaker stops the waiting once the service is known bad. You need both.
- **"It makes my system slower."** The state check costs microseconds; against a 30 s timeout per call to a dead service, it makes failure handling thousands of times faster.

Part 14 — Check your understanding

1. Why does CLOSED mean traffic flows?
2. Why is a slow service more dangerous than a completely dead one?
3. What two separate benefits does an open circuit provide?
4. Why does Half-Open allow only a *few* requests through?
5. What's the hidden risk of a circuit breaker working well?
6. Why does the example's `synchronized` become a problem at scale?

(Answers: 1 — it's an electrical metaphor; a closed circuit is a complete loop that current flows through; 2 — a dead service refuses instantly so callers move on, while a slow one holds caller threads hostage until timeout; 3 — the caller stops wasting resources waiting, and the failing service stops receiving load so it can recover; 4 — to avoid overwhelming a service that has only just started recovering; 5 — it suppresses the errors that would normally alert you, so problems go unnoticed without explicit alerting on breaker state; 6 — every call acquires the same lock, so the breaker itself becomes a bottleneck under high concurrency.)

Part 15 — Interview soundbite (30 seconds)

"A circuit breaker stops you repeatedly calling a dependency that's already failing. The problem is cascading failure: if B hangs, A's threads block waiting on it, and A runs out of threads and dies even though its own code is fine. Slow is worse than down, because a dead service refuses instantly while a slow one holds your threads hostage. The breaker has three states — Closed, where traffic flows and failures are counted; Open, once failures cross a threshold, where calls are rejected instantly with a fallback; and Half-Open, after a timeout, where a few trial requests probe recovery. Success closes it, failure re-opens it, and only a few requests pass so you don't re-kill a service just getting back up. It does two things: your service stops wasting resources, and the failing service stops receiving load so it can recover — breaking the vicious circle where retry traffic prevents recovery. Netflix's Hystrix is the classic: if recommendations is down they still stream video and drop that row. Tuning matters — too sensitive and it flaps, too lenient and it doesn't protect — so use a percentage threshold over a rolling window, not a raw count. And alert on breaker state, because a breaker working well hides the errors that would tell you something's broken."

Part 16 — Implementation

A. Runnable demo — watch the state machine work

(A **state machine** is always in exactly one of a fixed set of states, moving between them on defined events — exactly what the breaker is.)

Full source: `demos/chapters/06-circuit-breaker/CircuitBreakerDemo.java`

```
cd demos && ./gradlew runCh06
```

The demo's breaker is the Part 7 class with two changes: `call(action, fallback)` takes a **fallback** that is returned both when the circuit rejects a call and when a call fails, and **a failed half-open probe re-opens immediately** (`state == HALF_OPEN || failureCount >= threshold`). It also logs each transition. A `FlakyService` (thread-safe `AtomicInteger` call counter, `volatile` health flag visible to all threads) is healthy, then broken, then healed; `main` drives four phases with a `"payments"` breaker of 3 failures / 2 s open:

```
synchronized <T> T call(Callable<T> action, Callable<T> fallback) throws Exception {
    if (state == State.OPEN) {
        if (System.currentTimeMillis() - lastFailureTime < openTimeoutMs) {
            log("REJECTED (circuit OPEN) -> fallback");
            return fallback.call();          // fail fast, no remote call
        }
        transitionTo(State.HALF_OPEN);       // timeout elapsed: allow a probe
    }
    try {
        T result = action.call();
        failureCount = 0;
        transitionTo(State.CLOSED);
        return result;
    } catch (Exception e) {
        failureCount++;
        lastFailureTime = System.currentTimeMillis();
        if (state == State.HALF_OPEN || failureCount >= failureThreshold) {
            transitionTo(State.OPEN);
        }
        return fallback.call();
    }
}
```

B. What you should see

```
--- Phase 1: service healthy ---
  result = real-data          (x3, circuit stays CLOSED)

--- Phase 2: service breaks ---
    [payments] call failed (1/3): service unavailable
    [payments] call failed (2/3): service unavailable
    [payments] call failed (3/3): service unavailable
    [payments] state CLOSED -> OPEN
    [payments] REJECTED (circuit OPEN) -> fallback
  >> service was actually called 6 times so far

--- Phase 3: still broken, circuit stays open ---
  >> extra real calls during OPEN: 0

--- Phase 4: wait for timeout, service recovers ---
    [payments] state OPEN -> HALF_OPEN
    [payments] probe SUCCEEDED
    [payments] state HALF_OPEN -> CLOSED
  result = real-data
```

The line that proves the pattern is `extra real calls during OPEN: 0` **.** Three requests came in; the failing service received none — the caller didn't wait, and the sick service was left alone.

C. Production — Resilience4j

Nobody hand-writes breakers in production. **Resilience4j** is the current Java standard (Hystrix is in maintenance mode). This is the **one place the book shows its setup**; the Bulkhead and Retry chapters reuse it and only show their own config block.

Dependency (Maven): Spring Boot is the framework for building Java services; **AOP** (aspect-oriented programming) is what lets an annotation like `@CircuitBreaker` wrap a method with extra behaviour, so it must be on the classpath:

```
<dependency>
  <groupId>io.github.resilience4j</groupId>
  <artifactId>resilience4j-spring-boot3</artifactId>
  <version>2.2.0</version>
</dependency>
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-aop</artifactId>
</dependency>
```

Configuration (`application.yml`) — where the real learning is (`400` = HTTP "bad request"):

```
resilience4j:
  circuitbreaker:
    instances:
      paymentService:
        # Use a percentage over a rolling window, not a raw count
        slidingWindowType: COUNT_BASED
        slidingWindowSize: 20              # consider the last 20 calls
        minimumNumberOfCalls: 10           # don't judge until we have 10 samples
        failureRateThreshold: 50           # open if >50% of them failed
        waitDurationInOpenState: 10s       # how long to stay OPEN
        permittedNumberOfCallsInHalfOpenState: 3
        automaticTransitionFromOpenToHalfOpenEnabled: true
        # Treat slow calls as failures too - crucial, see pitfall 3
        slowCallRateThreshold: 50
        slowCallDurationThreshold: 2s
        recordExceptions:
          - java.io.IOException
          - java.util.concurrent.TimeoutException
        ignoreExceptions:
          - com.example.ValidationException   # never trip on a 400
```

Usage:

```
@Service
public class PaymentClient {

    @CircuitBreaker(name = "paymentService", fallbackMethod = "paymentFallback")
    public PaymentResult charge(Order order) {
        return restClient.post()
                .uri("/payments")
                .body(order)
                .retrieve()
                .body(PaymentResult.class);
    }

    // Signature must match, plus a trailing Throwable
    private PaymentResult paymentFallback(Order order, Throwable t) {
        log.warn("Payment circuit open for order {}: {}", order.getId(), t.toString());
        return PaymentResult.deferred(order.getId());   // queue for later
    }
}
```

Monitoring — expose breaker state so you can alert on it:

```
management:
  endpoints.web.exposure.include: health,circuitbreakers,prometheus
  health.circuitbreakers.enabled: true
```

Then alert on the **Prometheus** (open-source metrics collection and alerting) metric `resilience4j_circuitbreaker_state`.

D. Implementation pitfalls

1. Raw failure count instead of a rate over a window. Three failures scattered over an hour shouldn't trip anything. Use `slidingWindowSize` + `failureRateThreshold`, and `minimumNumberOfCalls` so a single early failure (100% rate) can't open the circuit.

2. Counting business errors as failures. A 400 or validation exception means the *request* was bad, not the *service*. Put them in `ignoreExceptions` or a client bug takes down a healthy dependency.

3. Forgetting `slowCallDurationThreshold`. A service returning `200 OK` after 30 s never "fails", so a failure-only breaker never opens — while your threads are consumed as badly as if it were down. Slow-call detection catches the most dangerous case.

4. No timeout on the underlying call. The breaker only reacts to calls that *return*. Set connect and read timeouts on the HTTP client, or a hung **socket** blocks forever and the breaker never counts a failure.

5. One breaker shared across dependencies. Name breakers per downstream service; shared, a reviews failure blocks payments.

6. Fallbacks that call other services. The fallback runs when things are already broken: keep it local and fast — cache, default, or queue. A networked fallback is a second failure waiting to happen.

7. Not alerting on state. The one people skip: a breaker doing its job hides errors. Alert on the open transition or you'll hear from a customer.

Related patterns

- **Bulkhead** — isolates resource pools; the breaker limits *how long* you try, the bulkhead limits *how much* damage one failure can do
- **Retry** — handles transient faults; the breaker stops retries becoming an attack on a downed service
- **Service Discovery** — per-instance breakers across a discovered pool
- **API Gateway** — a natural place to configure breaker rules per backend route
- **Sidecar** — service meshes provide breakers at the infrastructure level, with no application code